The Transformer Architecture

A Deep Dive into "Attention Is All You Need"

Vaswani et al., 2017 · NeurIPS

Introduction

The Paradigm Shift

The Paper That Changed Everything

"Attention Is All You Need" (Vaswani et al., 2017) introduced the Transformer —a sequence-to-sequence architecture built **entirely on attention** , eliminating recurrence and convolutions.

1. ### Eliminated Sequential Processing
   Unlike RNNs, Transformers process entire sequences in parallel, dramatically speeding up training.

2. ### Long-Range Dependencies
   Self-attention allows any token to directly attend to any other, regardless of distance.

3. ### State-of-the-Art Results
   Achieved superior translation quality with lower training costs than RNN/CNN-based models.

Core Innovation

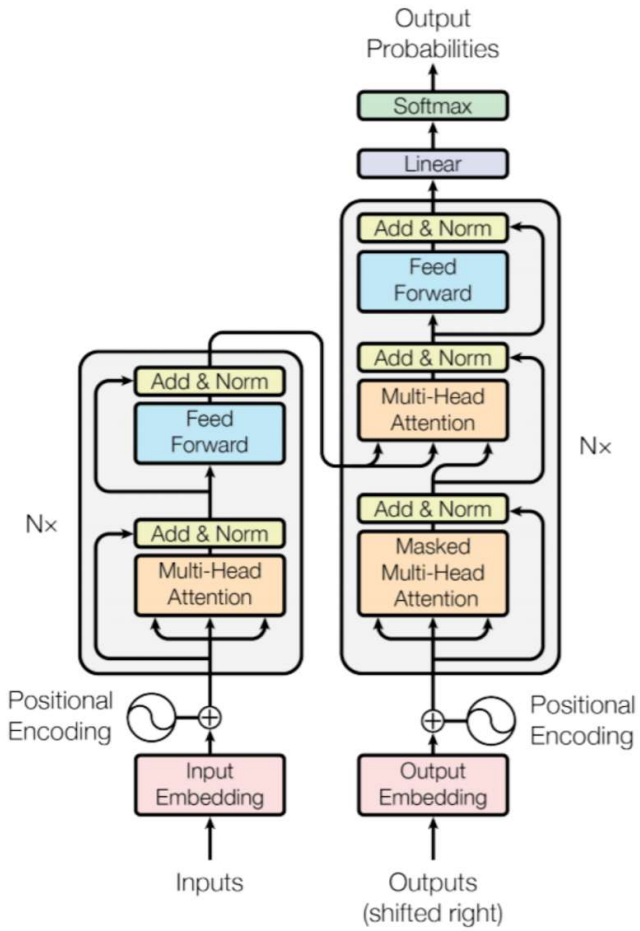
"A Transformer reads a whole sentence at once and lets each word decide which other words matter to it."

"Deciding who matters" = Attention
"At once" = Parallel Processing

> The Transformer became the foundation for BERT, GPT, T5, and modern Large Language Models

Architecture Overview

The Complete Transformer



Input Pipeline

Raw text → Tokenization → Embeddings (512-dim) + Positional Encoding

Encoder Stack (N=6)

Multi-Head Self-Attention + Feed-Forward Network. Each sublayer has residual connections & layer normalization.

Decoder Stack (N=6)

Masked Self-Attention + Cross-Attention + Feed-Forward. Generates output autoregressively.

Output Pipeline

Linear projection → Softmax → Probability distribution over vocabulary

Input Processing

Embeddings & Positional Encoding

From Text to Vectors

1. **Tokenization**
   Split text into tokens (words/subwords) using Byte-Pair Encoding (BPE)
2. **Embedding Lookup**
   Map each token to a dense vector (d_model = 512) via learned embedding matrix
3. **Add Positional Encoding**
   Inject position information since Transformers are permutation-invariant

Sinusoidal Positional Encoding

Uses sine and cosine functions of different frequencies to encode position:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

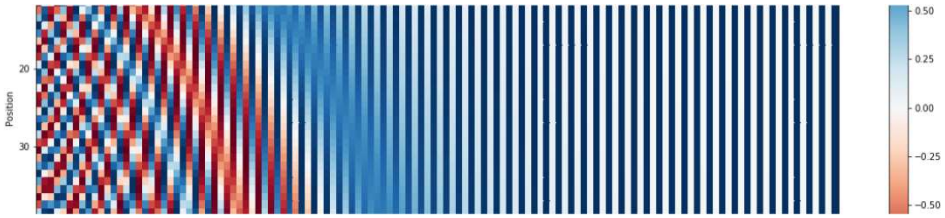
$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

pos = position in sequence, **i** = dimension index

- Allows the model to learn to attend by **relative positions**
- Can extrapolate to **sequence lengths** longer than training
- Each dimension has a **unique wavelength** forming a geometric progression

Final Input Representation

Input = Token Embedding + Positional Encoding



Core Mechanism

Self-Attention: Scaled Dot-Product

The Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Q (Query): What am I looking for?
K (Key): What do I contain?
V (Value): What information do I provide?

Step-by-Step Computation

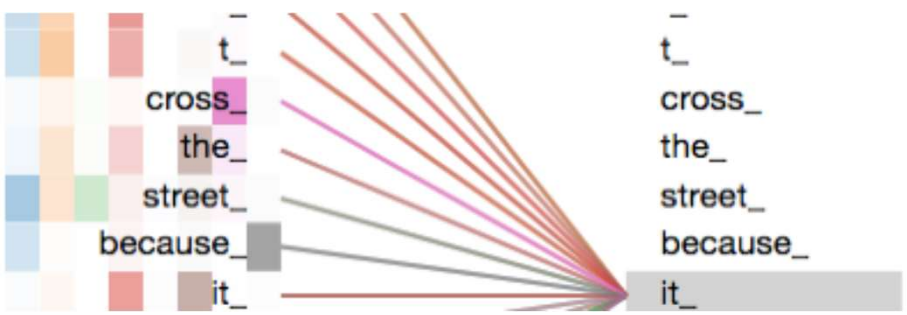
1. **Compute Attention Scores**
   Calculate dot products: $Q \times K^T$ (measures similarity)
2. **Scale by $\sqrt{d_k}$**
   Divide by $\sqrt{d_k}$ ($d_k = 64$) to **prevent softmax saturation**
3. **Apply Softmax**
   Normalize to get attention weights (sum to 1)
4. **Weighted Sum of Values**
   Multiply weights by V to get context-aware output

Why Scale by $\sqrt{d_k}$?

- For large d_k, dot products grow **magnitude**
- Large values push softmax into regions with **extremely small gradients**
- Scaling keeps variance ≈ 1, ensuring **stable gradients** during training

Example: For $d_k = 64$, $\sqrt{d_k} = 8$

Raw score: 112 → Scaled: 14



Each token attends to all other tokens, with attention weights indicating relevance

Enhanced Attention

Multi-Head Attention

Parallel Attention Heads

Instead of single attention, the model uses **h = 8 parallel attention heads** , each learning different types of relationships.

Multi-Head Formula

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W_O$$

where $\text{head}_i = \text{Attention}(QW_{iQ}, KW_{iK}, VW_{iV})$

Key Benefits

- **Multiple representation subspaces** — each head captures different linguistic patterns
- **Parallel computation** — all heads process simultaneously for efficiency
- **Enhanced expressiveness** — model can focus on different positions simultaneously

Architecture Details

Dimensions (per head):

$d_k = d_v = d_{model} / h = 512 / 8 = 64$

Weight Matrices:

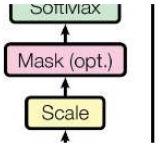
$W_Q, W_K, W_V \in \mathbb{R}^{(d_{model} \times d_k)}$

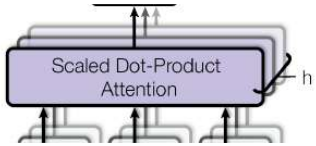
Output Projection:

$W_O \in \mathbb{R}^{(d_{model} \times d_{model})}$

Total Parameters:

$4 \times d_{model}^2 = 4 \times 512^2 \approx 1M$





Insight: Different heads learn different linguistic phenomena—some focus on syntax, others on semantics, coreference, or positional relationships.

Processing Layer

Feed-Forward Networks

Position-Wise FFN

After attention aggregates information across positions, the **Feed-Forward Network (FFN)** processes each position independently to transform the representations.

FFN Formula

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Two linear transformations with ReLU activation
- Applied identically to each position

The Expansion-Compression Pattern

Input:	512-dim
↓	
Hidden Layer:	2048-dim (4× expansion)
↓	
Output:	512-dim

Why Expand Then Compress?

- **Higher-dimensional space** provides more capacity for complex, non-linear transformations
- Like spreading work on a large table (2048-dim) to do complex operations, then packing neatly (512-dim)
- The FFN adds **significant learning capacity** and non-linearity to the model

Surprising Fact

For BERT-base (110M parameters):

Attention layers:	~25M (22%)
FFN layers:	~75M (68%)
Embeddings + other:	~10M (10%)

The FFN contains **most of the model's parameters** —it's doing the heavy lifting!

> **Analogy:** Attention = Communication between tokens | FFN = Computation within each token

Training Stability

Layer Normalization & Residuals

Residual Connections (Skip Connections)

Residual connections add the input of a sublayer to its output, creating a **"shortcut" path** that helps gradients flow through deep networks.

Residual Connection Formula

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Add** input x to sublayer output
- **Normalize** the result

Benefits

- **Prevents vanishing gradients** in deep networks (12-24+ layers)
- Allows the network to learn **incremental changes** rather than full transformations
- **Preserves information** from earlier layers
- Enables training of **much deeper models**

Layer Normalization

Layer Normalization **stabilizes training** by normalizing inputs across the feature dimension, ensuring consistent value ranges.

How It Works

1. Calculate **mean μ** and **variance σ^2** for each sample
2. **Normalize** : $\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$
3. **Scale and shift** : $y = \gamma\hat{x} + \beta$ (learnable parameters)

Two Variants

Post-LN (Original)

x + Sublayer(LayerNorm(x))

Used in original Transformer

Pre-LN (Modern)

LayerNorm(x + Sublayer(x))

Better for deep models (GPT-3)

Dropout (p=0.1) is applied after each sublayer for regularization, preventing overfitting.

Encoder Architecture

The Encoder Stack

Encoder Layer Structure

The encoder consists of **N = 6 identical layers** (in the original paper). Each layer has two sublayers with residual connections and layer normalization.

Encoder Layer Flow

1. **Multi-Head Self-Attention**
   All tokens attend to all tokens (bidirectional)
2. **Add & Norm (Residual + LayerNorm)**
   Stabilize and preserve information
3. **Feed-Forward Network**
   Position-wise transformation (512→2048→512)
4. **Add & Norm (Residual + LayerNorm)**
   Final normalization before next layer

Key Property: The encoder is **non-autoregressive** —all positions are processed simultaneously, enabling full parallelization.

What the Encoder Learns

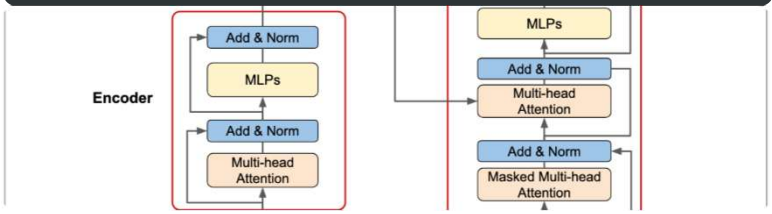
- **Contextualized representations** —each token's embedding is enriched by its context
- **Long-range dependencies** —tokens can directly attend to distant relevant tokens
- **Syntactic & semantic patterns** —attention heads learn grammar, coreference, etc.

Encoder Output

After N layers, the encoder produces:

Contextualized representations: $\mathbb{R}$^(seq_len × d_model)

These are passed to the decoder via cross-attention.



Decoder Architecture

The Decoder Stack

Decoder Layer Structure

The decoder has **N = 6 identical layers** , each with **three sublayers** —one more than the encoder for cross-attention to the encoder output.

Decoder Layer Flow

1. **Masked Multi-Head Self-Attention**
   Prevents attending to future positions (autoregressive)
2. **Add & Norm**
   Residual connection + LayerNorm
3. **Encoder-Decoder Cross-Attention**
   Decoder queries attend to encoder keys & values
4. **Add & Norm**
   Residual connection + LayerNorm
5. **Feed-Forward Network**
   Position-wise transformation
6. **Add & Norm**
   Final normalization

Masked Self-Attention (Causal Masking)

To maintain **autoregressive property** , the decoder uses a **causal mask** that prevents positions from attending to future positions.

Causal Mask (lower triangular):

```
[[1, 0, 0, 0],
[1, 1, 0, 0],
[1, 1, 1, 0],
[1, 1, 1, 1]]
```

Position i can only attend to positions **≤ i** , ensuring no "cheating" by looking ahead.

Cross-Attention Mechanism

- **Queries (Q)** come from decoder's previous layer
- **Keys (K)** and **Values (V)** come from encoder output
- This allows the decoder to **focus on relevant parts** of the input when generating each output token

Autoregressive Generation: During inference, tokens are generated one at a time, with each new token attending to all previously generated tokens.

Training & Applications

Training, Inference & Impact

Training Process

Teacher Forcing

During training, the decoder receives the **ground truth tokens** as input (not its own predictions), enabling parallel training.

Loss Function

Cross-entropy loss at each position:

$$L = -\Sigma \log P(y_t \mid y, x)$$

Optimization

Adam optimizer with learning rate warmup and decay. Label smoothing ($\varepsilon_{ls} = 0.1$) improves generalization.

Inference: Autoregressive Generation

At each step, the decoder generates one token, which is fed back as input for the next step until the end-of-sequence token is produced.

Transformer's Revolutionary Impact

Foundation for Modern NLP

BERT (encoder-only), **GPT** (decoder-only), **T5** (encoder-decoder)—all built on Transformer architecture.

Key Advantages

✓ **Parallelization** —train faster than RNNs

✓ **Scalability** —performance improves with size & data

✓ **Long-range dependencies** —O(1) path length between any two positions

✓ **Versatility** —applies to vision, speech, multimodal tasks

Beyond NLP

Vision
ViT, DETR, SAM

Multimodal
CLIP, GPT-4V, Gemini

Speech
Whisper, wav2vec 2.0

Science
AlphaFold, Graphormer

Key Takeaways

The Transformer's Lasting Legacy

1 Self-Attention

Enables **global dependencies** with O(1) path length. Each token directly attends to all others, capturing context regardless of distance.

2 Multi-Head Attention

Parallel attention heads capture diverse relationships. Each head learns different linguistic patterns, enhancing representational power.

3 Parallel Processing

Unlike RNNs, Transformers process **entire sequences at once**, enabling massive parallelization and faster training on modern hardware.

4 Residual Connections

Enable training of **very deep networks** (12-100+ layers) by preventing vanishing gradients and preserving information flow.

The Transformer architecture has become the foundation of modern AI, extending from NLP to computer vision, multimodal models, and scientific applications. Its core innovation—attention as the primary computation mechanism—continues to drive breakthroughs across domains.

> "The Transformer is to NLP what ResNet was to computer vision." — Andrew Ng